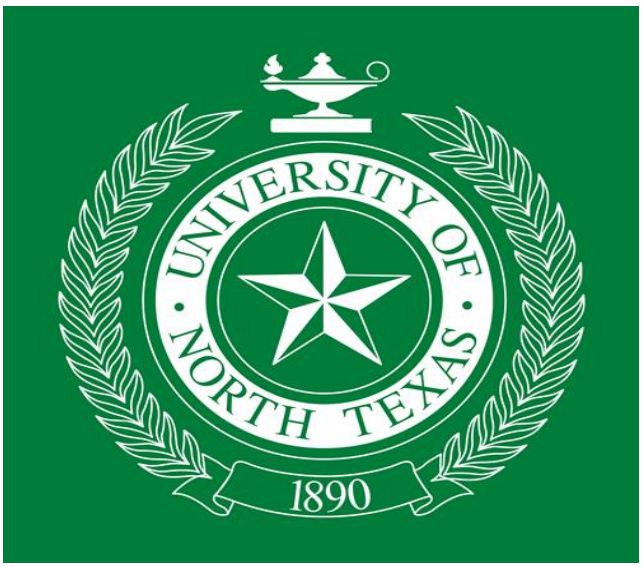
Project Report

GROUP - 1

Prediction of Calories Burnt Using Machine Learning

By

Shivanjani Acholu
Sravya Ravikanti
Shiva Priya Ummareddy
Suchitha Reddy Chinthireddy
Swathi Pasham



Department of Data Science

University of North Texas, Denton, Texas

Prediction of Calories burnt using Machine Learning

by

Shivanjani Acholu (Student ID: 11612390)
shivanjaniacholu@my.unt.edu
Major: Data Science
Role: Python programming, Project Documentation,
Domain Understanding & Feature Engineering,
Exploratory Data Analysis, Model Training, and Model Evaluation

Sravya Ravikanti (Student ID: 11632476)
sravyaravikanti@my.unt.edu
Major: Data Science
Role: Python programming, Data Visualization,
Exploratory Data Analysis, Data Pre-processing,
Parameter tuning and Model Training

Shiva Priya Ummareddy (Student ID: 11641605)
shivapriyaummareddy@my.unt.edu
Major: Data Science
Role: Python programming,
Dataset Selection, Parameter tuning and
Model Prediction and Evaluation

Swathi Pasham (Student ID: 11656634)
swathipasham@my.unt.edu
Major: Data Science
Role: Python programming, Data Visualization,
Data Pre-processing, Feature Engineering,
Algorithm Selection and Model Prediction

Suchitha Reddy Chinthireddy (Student ID:11636240)
suchithareddychinthireddy@my.unt.edu
Major: Data Science
Role: Python programming, Dataset selection,
Domain Understanding & Feature Engineering,
Parameter tuning and Model training

Workflow

1. *Data Collection:*
    - Collect the data on exercise and calories burned from the given link.
    - Load the data into a pandas data frame.

2. *Exploratory Data Analysis (EDA):*
    - Perform EDA to understand the data and identify any issues or anomalies.
    - Check for missing values and handle them appropriately.
    - Identify and handle outliers using boxplots and scatterplots.
    - Check for zero correlation between variables and remove them.
    - Visualize the data using plots and histograms to get insights.

3. *Data Processing and Feature Engineering:*
    - Prepare the data for modelling by performing feature engineering and data processing.
    - Remove the user id column as it does not provide any value for the model.
    - Normalize the data using log square root transformation.
    - Perform scaling using Min-Max and Max Absolute methods.
    - Compute the correlation matrix and identify highly correlated features for feature selection.

4. *Model Building:*
    - Build a machine learning model for calories prediction using exercise.
    - Split the data into training and testing sets.
    - Train the model using regression techniques like Linear Regression, Ridge Regression, Lasso Regression. Evaluate the performance of the model using R squared and adjust the model as necessary.
    - Identify the most important features using the regressor coefficients.

5. *Model Deployment:*
    - Deploy the model to a production environment for use by end-users.
    - Provide instructions for using the model and the appropriate input data format.

Abstract

The field of data science involves the extraction of meaningful insights and knowledge from data through the use of various statistical and computational techniques. The objective of data science is to make informed decisions and predictions based on data-driven analysis. With the increasing popularity of digital devices and the internet of things, there is an abundance of data that can be used to solve real-world problems.

In this context, one of the most popular applications of data science is predicting calorie consumption using machine learning regression models. This involves analysing data from various factors such as age, height, weight, duration of exercise, heart rate, and body temperature to predict the number of calories burned during physical activity.

In this project, we explore the process of building a basic machine learning regression model to predict calorie consumption. We start with data collection and cleaning, followed by exploratory data analysis to identify important variables and relationships. We then implement a basic machine learning regression model using scikit-learn in Python and evaluate the model's performance using relevant metrics such as mean squared error and R-squared.

Through this project, we gain a better understanding of the fundamentals of data science and how it can be applied to solve real-world problems such as calorie prediction. By using machine learning regression models, we can make accurate predictions based on large datasets, helping individuals make informed decisions about their physical activity and diet.

Data Specification

Based on the provided dataset, it appears that the project involves building a calorie prediction model using supervised machine learning techniques. The dataset contains various features related to physical activity, such as Gender, Age, Height, Weight, Duration, Heart Rate, and Body Temperature, as well as the corresponding calorie consumption values.

The goal of the project is to use this dataset to train a machine learning model that can accurately predict the calorie consumption based on the given features. This could be useful for individuals who are trying to manage their weight and monitor their physical activity levels.

The dataset consists of a features x observations matrix with 5 features (Gender, Age, Height, Weight, Duration, Heart Rate, and Body Temperature) and 15000 observations (or samples). Each row represents a different physical activity session, with the corresponding values for the different features and the calorie consumption.

Project design & Modelling

In this project, we aim to predict the value of y based on the independent variables Gender, Age, Height, Weight, Duration, Heart_Rate, and Body_Temp. To accomplish this, we used Python and several libraries including NumPy, Pandas, Matplotlib, and Scikit-learn.

We used multiple linear regression models to predict the value of y. The first model we used was the ordinary least squares (OLS) regression, which is a method of estimating the unknown parameters in a linear regression model. We also used the Ridge regression model and the Lasso regression model to improve the performance of our model.

The design of our program involved several important decisions. We decided to pre-process the data by removing any missing values, scaling the features using the Standard Scaler, and splitting the data into training and testing sets. We also decided to perform feature selection using the Recursive Feature Elimination (RFE) method to determine which features were most important in predicting the value of y.

The most important functions and/or methods in our project include the following:

- read_csv(): This function is used to read in the data from a CSV file and store it in a Pandas DataFrame.
- dropna(): This method is used to remove any missing values from the DataFrame.
- StandardScaler(): This class is used to scale the features in the DataFrame.
- train_test_split(): This function is used to split the data into training and testing sets.
- LinearRegression(): This class is used to create an OLS regression model.
- Ridge(): This class is used to create a Ridge regression model.
- Lasso(): This class is used to create a Lasso regression model.
- fit(): This method is used to train the model on the training data.
- predict(): This method is used to predict the value of y based on the testing data.

The functionality of our program is straightforward. The user inputs the data for Gender, Age, Height, Weight, Duration, Heart_Rate, and Body_Temp, and the program outputs a predicted value for y based on the regression model that was trained on the training data. If the user inputs invalid data, such as a non-numeric value for Age or Height, the program will display an error message and ask the user to input valid data.

Group1_Project_Report_5502.ipynb

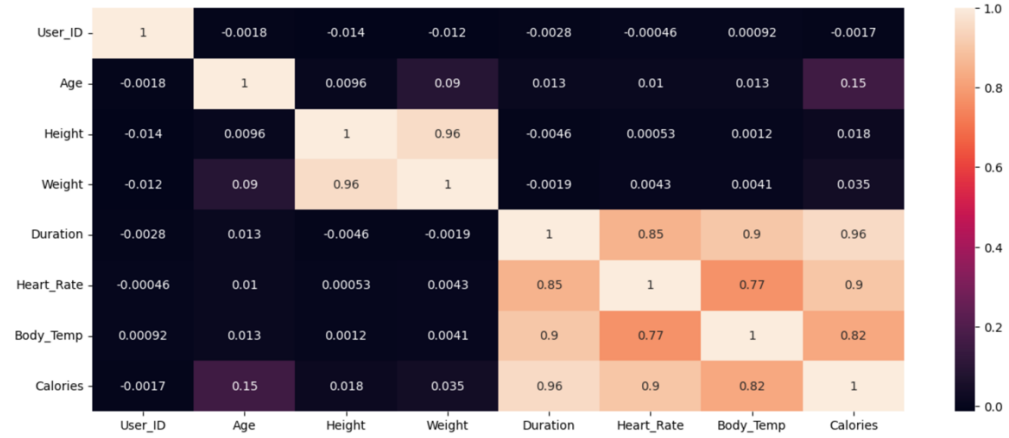
File Edit View Insert Runtime Tools Help All changes saved

Comment Share

+ Code + Text

```
#Visualization of the correlation between the different columns in the dataset.
plt.figure(figsize=(15,6))
sns.heatmap(joined_data.corr(),annot=True)
plt.show()
```

```
<ipython-input-12-ea1390b0fb26>:3: FutureWarning: The default value of numeric_only in DataFrame.corr is deprecated. In a future version, it will default to False. S
  sns.heatmap(joined_data.corr(),annot=True)
```



Group1_Project_Report_5502.ipynb

File Edit View Insert Runtime Tools Help All changes saved

Comment Share

+ Code + Text

3	16180408	female	34	179.0	71.0	13.0	100.0	40.5	71.0
4	17771927	female	27	154.0	58.0	10.0	81.0	39.8	35.0

```
# Create a new feature for body mass index (BMI)
joined_data["BMI"] = joined_data["Weight"] / (joined_data["Height"] / 100) ** 2

# Create a new feature for heart rate percentage of maximum heart rate
joined_data["Heart_Rate_Percentage"] = joined_data["Heart_Rate"] / (220 - joined_data["Age"])

# Create a new feature for duration in minutes
joined_data["Duration_Minutes"] = joined_data["Duration"] / 60

# Create a new feature for total energy expenditure (TEE)
joined_data["TEE"] = joined_data["Calories"] + (joined_data["Duration_Minutes"] * 5.0) + (joined_data["BMI"] * 10.0)

# Create a new feature for age groups
bins = [0, 18, 30, 45, 60, 120]
labels = ["<18", "18-30", "30-45", "45-60", ">60"]
joined_data["Age_Group"] = pd.cut(joined_data["Age"], bins=bins, labels=labels)

# Create a new feature for BMI groups
bins = [0, 18.5, 25, 30, 100]
labels = ["Underweight", "Normal", "Overweight", "Obese"]
joined_data["BMI_Group"] = pd.cut(joined_data["BMI"], bins=bins, labels=labels)
```

```
[16] joined_data.head()
```

	User_ID	Gender	Age	Height	Weight	Duration	Heart_Rate	Body_Temp	Calories	BMI	Heart_Rate_Percentage	Duration_Minutes	TEE	Age_Group	BMI_Group
0	14733363	male	68	190.0	94.0	29.0	105.0	40.8	231.0	26.038781	0.690789	0.483333	493.804478	>60	Overweight
1	14861698	female	20	166.0	60.0	14.0	94.0	40.3	66.0	21.773842	0.470000	0.233333	284.905090	18-30	Normal

We have also created a calorie prediction app. The app requires the user to input their age, height, weight, duration of activity, heart rate, and body temperature in order to predict the number of calories burned during the activity. The form includes labels for each input field and the "Predict Calories" button at the bottom of the form. The webpage is displayed with a title of "Calorie Prediction App" and a header of "Calorie Prediction App" in the body of the page.

Project Results:

Model Scores

```
Linear regression Model Score  1.173687756066081
Ridge regression Model Score  1.1714478959341745
Lasso regression Model Score  1.1239976261453863
SVR regression Model Score  1.2213656895381537
XGB regression Model Score  1.2461755934557024
Decision Tree regression Model Score  1.179678740136945
```

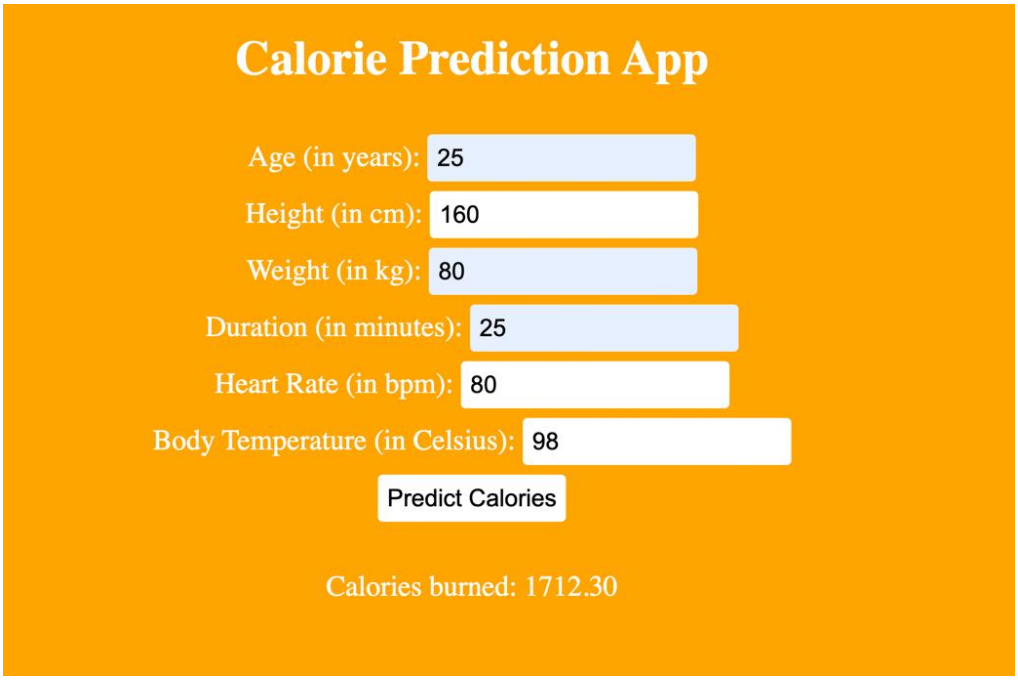
MSE Values

```
Linear regression Model Score  -0.08496779955782685
Ridge regression Model Score  -0.0828972521689908
Lasso regression Model Score  -0.039033784619731504
SVR regression Model Score  -0.12904172151816362
XGB regression Model Score  -0.15197622579457049
Decision Tree regression Model Score  -0.09050592055377038
```

Adjusted R2 Values

```
Linear regression Model Score  1.173687756066081
Ridge regression Model Score  1.1714478959341745
Lasso regression Model Score  1.1239976261453863
SVR regression Model Score  1.2213656895381537
XGB regression Model Score  1.2461755934557024
Decision Tree regression Model Score  1.179678740136945
```

UI Result:



References

- GitHub: https://github.com/Kasra1377/calories-burned-prediction/blob/main/calories-prediction-streamlit.py
- Kaggle : https://www.kaggle.com/code/fmendes/exercise-and-calories
- Stack overflow: https://stackoverflow.com/questions/17071871/how-do-i-select-rows-from-a-dataframe-based-on-column-values/17071908#17071908
- YouTube: https://www.youtube.com/watch?v=VqgUkExPvLY&ab_channel=CodingIsFun

ANNEXURE:

Code for Project:

In [1]:
```
import pandas as pd
```

In [2]:
```
#Reading Dataset
exercise_data = pd.read_csv( 'exercise.csv' )
```

In [3]:
```
exercise_data.head()
```

Out[3]:

	User_ID	Gender	Age	Height	Weight	Duration	Heart_Rate	Body_Temp
0	14733363	male	68	190.0	94.0	29.0	105.0	40.8
1	14861698	female	20	166.0	60.0	14.0	94.0	40.3
2	11179863	male	69	179.0	79.0	5.0	88.0	38.7
3	16180408	female	34	179.0	71.0	13.0	100.0	40.5
4	17771927	female	27	154.0	58.0	10.0	81.0	39.8

In [4]:
```
exercise_data.describe()
```

Out[4]:

	User_ID	Age	Height	Weight	Duration	Heart_Rate	1
count	1.500000e+04	15000.000000	15000.000000	15000.000000	15000.000000	15000.000000	
mean	1.497736e+07	42.789800	174.465133	74.966867	15.530600	95.518533	
std	2.872851e+06	16.980264	14.258114	15.035657	8.319203	9.583328	
min	1.000116e+07	20.000000	123.000000	36.000000	1.000000	67.000000	
25%	1.247419e+07	28.000000	164.000000	63.000000	8.000000	88.000000	
50%	1.499728e+07	39.000000	175.000000	74.000000	16.000000	96.000000	
75%	1.744928e+07	56.000000	185.000000	87.000000	23.000000	103.000000	
max	1.999965e+07	79.000000	222.000000	132.000000	30.000000	128.000000	

In [5]:
```
calories_data = pd.read_csv( 'calories.csv' )
```

In [6]:
```
calories_data.describe()
```

Out[6]:

	User_ID	Calories
count	1.500000e+04	15000.000000
mean	1.497736e+07	89.539533
std	2.872851e+06	62.456978
min	1.000116e+07	1.000000
25%	1.247419e+07	35.000000
50%	1.499728e+07	79.000000
75%	1.744928e+07	138.000000
max	1.999965e+07	314.000000

In [7]:
```
# joining both CSV files using User_ID as key and left outer join
joined_data = exercise_data.join( calories_data.set_index( 'User_ID' ) )
joined_data.head()
```

Out[7]:

	User_ID	Gender	Age	Height	Weight	Duration	Heart_Rate	Body_Temp	Calories
0	14733363	male	68	190.0	94.0	29.0	105.0	40.8	231.0
1	14861698	female	20	166.0	60.0	14.0	94.0	40.3	66.0
2	11179863	male	69	179.0	79.0	5.0	88.0	38.7	26.0
3	16180408	female	34	179.0	71.0	13.0	100.0	40.5	71.0
4	17771927	female	27	154.0	58.0	10.0	81.0	39.8	35.0

In [8]:
```
# Check for null values
print(joined_data.isnull().sum())
```

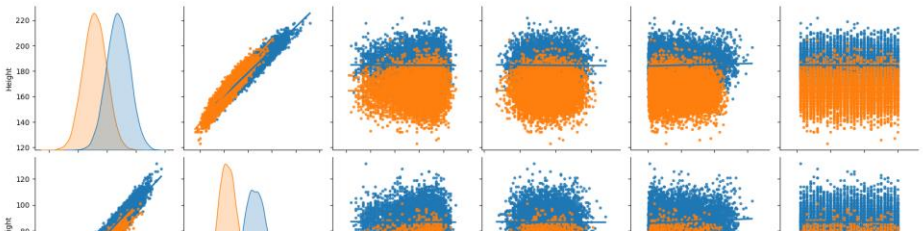
```
User_ID       0
Gender        0
Age           0
Height        0
Weight        0
Duration      0
Heart_Rate    0
Body_Temp     0
Calories      0
dtype: int64
```

In [9]:
```
# Check for missing values
print(joined_data.isna().sum())
```

```
User_ID       0
Gender        0
Age           0
Height        0
Weight        0
Duration      0
Heart_Rate    0
Body_Temp     0
Calories      0
dtype: int64
```

In [10]:
```
# matrix of scatter plot graphs
%matplotlib inline

import seaborn as sns

cols = [ "Height", "Weight", "Body_Temp", "Heart_Rate", "Calories", "D

# shows gender in different colors and a linear regression model for ea
plot = sns.pairplot( joined_data[ cols ], size=3, markers=".", hue="Ge

# get x labels from the last row and save them into an array
xlabels = []
for ax in plot.axes[ -1, : ]:
    xlabel = ax.xaxis.get_label_text()
    xlabels.append( xlabel )

# apply x labels from the array to the rest of the graphs
y_ax_len = len( plot.axes[ :, 0 ] )
for i in range( len( xlabels ) ):
    for j in range( y_ax_len ):
        if j != i :
            plot.axes[ j, i ].xaxis.set_label_text( xlabels[ i ] )
```

```
/Users/shivanjanireddy/opt/anaconda3/lib/python3.9/site-packages/seab
orn/axisgrid.py:2076: UserWarning: The `size` parameter has been rena
med to `height`; please update your code.
  warnings.warn(msg, UserWarning)
```



In [12]:
```
#Visualization of the correlation between the different columns in the
plt.figure(figsize=(15,6))
sns.heatmap(joined_data.corr(),annot=True)
plt.show()
```

In [13]:
```
#Feature Engineering
```

In [14]:
```
joined_data.head()
```

Out[14]:

	User_ID	Gender	Age	Height	Weight	Duration	Heart_Rate	Body_Temp	Calories
0	14733363	male	68	190.0	94.0	29.0	105.0	40.8	231.0
1	14861698	female	20	166.0	60.0	14.0	94.0	40.3	66.0
2	11179863	male	69	179.0	79.0	5.0	88.0	38.7	26.0
3	16180408	female	34	179.0	71.0	13.0	100.0	40.5	71.0
4	17771927	female	27	154.0	58.0	10.0	81.0	39.8	35.0

In [15]:
```
# Create a new feature for body mass index (BMI)
joined_data["BMI"] = joined_data["Weight"] / (joined_data["Height"] /

# Create a new feature for heart rate percentage of maximum heart rate
joined_data["Heart_Rate_Percentage"] = joined_data["Heart_Rate"] / (22

# Create a new feature for duration in minutes
joined_data["Duration_Minutes"] = joined_data["Duration"] / 60

# Create a new feature for total energy expenditure (TEE)
joined_data["TEE"] = joined_data["Calories"] + (joined_data["Duration_

# Create a new feature for age groups
bins = [0, 18, 30, 45, 60, 120]
labels = ["<18", "18-30", "30-45", "45-60", ">60"]
joined_data["Age_Group"] = pd.cut(joined_data["Age"], bins=bins, label

# Create a new feature for BMI groups
bins = [0, 18.5, 25, 30, 100]
labels = ["Underweight", "Normal", "Overweight", "Obese"]
joined_data["BMI_Group"] = pd.cut(joined_data["BMI"], bins=bins, label
```

In [16]:
```
joined_data.head()
```

Out[16]:

	User_ID	Gender	Age	Height	Weight	Duration	Heart_Rate	Body_Temp	Calories	
0	14733363	male	68	190.0	94.0	29.0	105.0	40.8	231.0	26.038
1	14861698	female	20	166.0	60.0	14.0	94.0	40.3	66.0	21.773
2	11179863	male	69	179.0	79.0	5.0	88.0	38.7	26.0	24.655
3	16180408	female	34	179.0	71.0	13.0	100.0	40.5	71.0	22.159
4	17771927	female	27	154.0	58.0	10.0	81.0	39.8	35.0	24.456

In [17]:
```
# Drop the User_ID column
joined_data.drop(columns=["User_ID"], inplace=True)
```

In [18]:
```
#Data Preprocessing
```

In [19]:
```
from sklearn.preprocessing import StandardScaler
```

In [20]:
```
# Convert Gender column to binary
joined_data["Gender"] = pd.get_dummies(joined_data["Gender"], drop_fir

# Fill missing values with mean
joined_data = joined_data.fillna(joined_data.mean())

# Standardize the data
scaler = StandardScaler()
joined_data[["Age","Height", "Weight","Duration", "Heart_Rate", "Body_
```

```
/var/folders/l7/y7mrsh9x3tddbxjdhc2jnr1r0000gn/T/ipykernel_2085/18569
837.py:5: FutureWarning: Dropping of nuisance columns in DataFrame re
ductions (with 'numeric_only=None') is deprecated; in a future versio
n this will raise TypeError.  Select only valid columns before callin
g the reduction.
  joined_data = joined_data.fillna(joined_data.mean())
```

In [21]:
```
# Log square root transformation
# Create the log square root transformations for the columns
joined_data['log_age'] = np.log(joined_data['Age'])
joined_data['sqrt_Height'] = np.sqrt(joined_data['Height'])
joined_data['log_weight'] = np.log(joined_data['Weight'])
joined_data['log_Duration'] = np.log(joined_data['Duration'])
joined_data['sqrt_Heart_Rate'] = np.sqrt(joined_data['Heart_Rate'])
joined_data['sqrt_Body_Temp'] = np.sqrt(joined_data['Body_Temp'])
joined_data['log_BMI'] = np.log(joined_data['BMI'])
```

```
/Users/shivanjanireddy/opt/anaconda3/lib/python3.9/site-packages/pand
as/core/arraylike.py:397: RuntimeWarning: invalid value encountered i
n log
  result = getattr(ufunc, method)(*inputs, **kwargs)
/Users/shivanjanireddy/opt/anaconda3/lib/python3.9/site-packages/pand
as/core/arraylike.py:397: RuntimeWarning: invalid value encountered i
n sqrt
  result = getattr(ufunc, method)(*inputs, **kwargs)
/Users/shivanjanireddy/opt/anaconda3/lib/python3.9/site-packages/pand
as/core/arraylike.py:397: RuntimeWarning: invalid value encountered i
n log
  result = getattr(ufunc, method)(*inputs, **kwargs)
/Users/shivanjanireddy/opt/anaconda3/lib/python3.9/site-packages/pand
as/core/arraylike.py:397: RuntimeWarning: invalid value encountered i
n log
  result = getattr(ufunc, method)(*inputs, **kwargs)
/Users/shivanjanireddy/opt/anaconda3/lib/python3.9/site-packages/pand
as/core/arraylike.py:397: RuntimeWarning: invalid value encountered i
n sqrt
  result = getattr(ufunc, method)(*inputs, **kwargs)
/Users/shivanjanireddy/opt/anaconda3/lib/python3.9/site-packages/pand
as/core/arraylike.py:397: RuntimeWarning: invalid value encountered i
n sqrt
  result = getattr(ufunc, method)(*inputs, **kwargs)
```

In [22]:
```
# Save the preprocessed data to a new CSV file
joined_data.to_csv("preprocessed_data.csv", index=False)
```

In [23]:
```
#Model Selection
```

In [24]:
```
# Split the dataset into input features and target variables
X = joined_data.drop(['Calories', 'Gender', 'Age_Group','BMI_Group'],
y = joined_data['Calories']
```

In [25]:
```
#Scaling the data
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_scaled
```

Out[25]:
```
array([[ 1.48472604,  1.08958204,  1.26590864, ...,  0.4406983 ,
         0.87126956,  1.0754102 ],
       [-1.34217934, -0.59372619, -0.99545805, ...,         nan,
        -0.79448192, -1.69915127],
       [ 1.5436199 ,  0.31806577,  0.26824686, ...,         nan,
                nan,  0.22897176],
       ...,
       [ 0.01237949, -1.08469109, -1.12847962, ...,         nan,
        -1.9682809 , -0.88846209],
       [ 2.07366466,  1.29999557,  1.465441  , ...,         nan,
                nan,  1.07671254],
       [ 1.19025673, -0.10276129,  0.26824686, ...,         nan,
        -0.02313643,  1.28663865]])
```

In [26]:
```
# Drop any rows with missing values in y
y.dropna(inplace=True)
```

In [27]:
```
# Drop any rows with missing values in X
X.dropna(inplace=True)
```

In [28]:
```

# Scale the input features using StandardScaler
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

In [29]:
```
# assuming X and y are numpy arrays
X = X[:617]
y = y[:617]

# split the data into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.
```

In [30]:
```
#Remove customer IDs from the data set
joined_data = joined_data.iloc[:,1:]

#Convertin the predictor variable in a binary numeric variable
joined_data ['Calories'].replace(to_replace='Yes', value=1, inplace=Tr
joined_data ['Calories'].replace(to_replace='No',  value=0, inplace=Tr

#Let's convert all the categorical variables into dummy variables
joined_data_dummies = pd.get_dummies(joined_data )
joined_data_dummies.head()
```

Out[30]:

	Age	Height	Weight	Duration	Heart_Rate	Body_Temp	Calories	BMI	H
0	1.484726	1.089582	1.265909	1.619127	0.989404	0.994023	2.265002	26.038781	
1	-1.342179	-0.593726	-0.995458	-0.183990	-0.158461	0.352342	-0.376905	21.773842	
2	1.543620	0.318066	0.268247	-1.265861	-0.784569	-1.701035	-1.017367	24.655910	
3	-0.517665	0.318066	-0.263839	-0.304198	0.467647	0.609015	-0.296847	22.159109	
4	-0.929922	-1.435380	-1.128480	-0.664821	-1.515029	-0.289338	-0.873263	24.456063	

5 rows × 27 columns

In [31]:
```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_scaled
```

Out[31]:
```
array([[ 0.85338375,  0.38405153,  0.49163881, ...,  0.28931847,
         0.63818005,  0.42413794],
       [ 0.09849088, -0.11402476,  0.49163881, ..., -1.7118789 ,
        -0.84728467,  1.35726267],
       [-1.41129485, -0.3630629 , -0.38602092, ...,  1.60028531,
         0.63818005, -0.1190276 ],
       ...,
       [-0.8451252 , -1.23469639, -1.59280306, ..., -2.27426139,
        -0.84728467, -1.41905236],
       [-1.22257163,  0.25953246, -0.27631346, ...,  0.62166016,
         0.94343193, -1.04133973],
       [ 0.00412928, -0.48758197, -0.27631346, ...,  1.06984572,
        -0.03835936,  0.37346708]])
```

In [32]:
```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_
```

In [33]:
```
from sklearn.linear_model import LinearRegression
model = LinearRegression()
```

In [34]:
```
model.fit(X_train, y_train)
```

Out[34]:
```
LinearRegression()
```

In [35]:
```
model.score(X_test, y_test)
```

Out[35]:
```
-0.08827535927533292
```

In [36]:
```
from sklearn.decomposition import PCA

pca = PCA(0.95)
X_pca = pca.fit_transform(X)
X_pca.shape
```

Out[36]:
```
(617, 1)
```

In [37]:
```
pca = PCA(n_components=10)
X_pca = pca.fit_transform(X)
X_pca.shape
```

Out[37]:
```
(617, 10)
```

In [38]:
```
pca.explained_variance_ratio_
```

Out[38]:
```
array([9.97494319e-01, 9.95663724e-04, 7.54967236e-04, 3.89857107e-04
,
       1.47437710e-04, 9.03951981e-05, 4.89824249e-05, 4.06694304e-05
,
       3.03274499e-05, 3.88836441e-06])
```

In [39]:
```
X_pca
```

Out[39]:
```
array([[ 6.55838410e+01, -2.08206207e-01, -5.39191944e-01, ...,
         3.16196843e-02, -2.51233058e-01,  5.03299063e-02],
       [-2.07659339e+01, -1.78610515e+00, -5.46604200e-01, ...,
         2.59961646e-01, -1.38412082e-01, -6.63985415e-02],
       [ 2.78551806e+01,  8.55505764e-01,  2.48965903e+00, ...,
        -3.78192881e-02,  2.80995502e-01, -7.14155386e-02],
       ...,
       [-1.03595765e+02,  3.72790637e+00, -9.31988615e-01, ...,
        -5.17895814e-01, -7.26235779e-01, -9.97576287e-02],
       [ 1.47645463e+00,  9.58661405e-01,  1.51439245e+00, ...,
        -2.17774245e-01, -1.37788232e-02, -4.27270910e-03],
       [ 4.57247978e+01,  3.05645307e-01, -1.43693155e-01, ...,
         1.26225547e-01, -1.54576226e-01, -2.66754130e-02]])
```

In [40]:
```
X_train_pca, X_test_pca, y_train, y_test = train_test_split(X_pca, y,
```

In [41]:
```
from sklearn.linear_model import LinearRegression
model = LinearRegression()
```

In [42]:
```
model.fit(X_train_pca, y_train)
```

Out[42]:
```
LinearRegression()
```

In [43]:
```
Lin_Reg_MS = model.score(X_test_pca, y_test)
Lin_Reg_MS
```

Out[43]:
```
-0.0673260467194885
```

In [44]:
```
Y_pred = model.predict(X_test_pca)
```

In [45]:
```
y_test
```

Out[45]:
```
236   -0.729159
481    0.519743
239    0.279569
24    -1.385632
47     0.823962
          ...
31     0.743904
295    1.432401
413   -1.257540
108    0.823962
212    0.711881
Name: Calories, Length: 124, dtype: float64
```

In [46]:
```
def adj_r_squared(y_true, y_pred, n_features):
    n = len(y_true)
    residuals = y_true - y_pred
    ss_res = np.sum(residuals**2)
    ss_tot = np.sum((y_true - np.mean(y_true))**2)
    r2 = 1 - ss_res / ss_tot
    adj_r2 = 1 - ((1 - r2) * (n - 1) / (n - n_features - 1))
    return adj_r2
```

In [47]:
```
Lin_Reg_AR2 = adj_r_squared(y_test, Y_pred, 2)
Lin_Reg_AR2
```

Out[47]: -0.08496779955782707

In [48]:
```
from sklearn.metrics import mean_squared_error
```

In [49]:
```
Lin_Reg_Mse = mean_squared_error(y_test,Y_pred)
Lin_Reg_Mse
```

Out[49]: 1.1736877560660812

In [50]:
```
#Ridge Regression
from sklearn.linear_model import Ridge
```

In [51]:
```
ridge_model = Ridge(alpha=1)
```

In [52]:
```
ridge_model.fit(X_train_pca, y_train)
```

Out[52]: Ridge(alpha=1)

In [53]:
```
y_pred_ridge = ridge_model.predict(X_test_pca)
```

In [54]:
```
Ridge_Mse = mean_squared_error(y_test,y_pred_ridge)
Ridge_Mse
```

Out[54]: 1.1714478959341745

In [55]:
```
Ridge_MS = ridge_model.score(X_test_pca, y_test)
Ridge_MS
```

Out[55]: -0.06528916676786878

In [56]:
```
Ridge_AR2 = adj_r_squared(y_test, y_pred_ridge, 2)
Ridge_AR2
```

Out[56]: -0.0828972521689908

In [57]:
```
#Lasso Regression
from sklearn.linear_model import Lasso
```

In [58]:
```
lasso_model = Lasso(alpha=1.0)
```

In [59]:
```
lasso_model.fit(X_train_pca, y_train)
```

Out[59]: Lasso()

In [60]:
```
y_pred_lasso = lasso_model.predict(X_test_pca)
```

In [61]:
```
Lasso_Mse = mean_squared_error(y_test,y_pred_lasso)
Lasso_Mse
```

Out[61]: 1.1239976261453863

In [62]:
```
Lasso_MS = lasso_model.score(X_test_pca, y_test)
Lasso_MS
```

Out[62]: -0.02213892633323189

In [63]:
```
Lasso_AR2 = adj_r_squared(y_test, y_pred_lasso, 2)
Lasso_AR2
```

Out[63]: -0.039033784619731504

In [64]:
```
#Support Vector Regression
from sklearn.svm import SVR
```

In [65]:
```
svr_model = SVR(kernel='linear', C=1.0, epsilon=0.1)
```

In [66]:
```
svr_model.fit(X_train_pca, y_train)
```

Out[66]: SVR(kernel='linear')

In [67]:
```
y_pred_SVR = svr_model.predict(X_test_pca)
```

In [68]:
```
SVR_Mse = mean_squared_error(y_test,y_pred_SVR)
SVR_Mse
```

Out[68]:
```
1.2213656895451017
```

In [69]:
```
SVR_MS = svr_model.score(X_test_pca, y_test)
SVR_MS
```

Out[69]:
```
-0.1106833195485768
```

In [70]:
```
SVR_AR2 = adj_r_squared(y_test, y_pred_SVR, 2)
SVR_AR2
```

Out[70]:
```
-0.12904172152458626
```

In [71]:
```
pip install xgboost
```

```
Collecting xgboost
  Downloading xgboost-1.7.5-py3-none-macosx_10_15_x86_64.macosx_11_0_
x86_64.macosx_12_0_x86_64.whl (1.8 MB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.8/1.8 MB 6.8 MB/s eta
0:00:0000:0100:01
Requirement already satisfied: scipy in /Users/shivanjanireddy/opt/an
aconda3/lib/python3.9/site-packages (from xgboost) (1.9.1)
Requirement already satisfied: numpy in /Users/shivanjanireddy/opt/an
aconda3/lib/python3.9/site-packages (from xgboost) (1.21.5)
Installing collected packages: xgboost
Successfully installed xgboost-1.7.5
Note: you may need to restart the kernel to use updated packages.
```

In [72]:
```
#XG Boost Regression
from xgboost import XGBRegressor
```

In [73]:
```
xgb_model = XGBRegressor(learning_rate=0.1, max_depth=3, n_estimators=
```

In [74]:
```
xgb_model.fit(X_train_pca, y_train)
```

Out[74]:
```
XGBRegressor(base_score=None, booster=None, callbacks=None,
             colsample_bylevel=None, colsample_bynode=None,
             colsample_bytree=None, early_stopping_rounds=None,
             enable_categorical=False, eval_metric=None, feature_type
s=None,
             gamma=None, gpu_id=None, grow_policy=None, importance_ty
pe=None,
             interaction_constraints=None, learning_rate=0.1, max_bin
=None,
             max_cat_threshold=None, max_cat_to_onehot=None,
             max_delta_step=None, max_depth=3, max_leaves=None,
             min_child_weight=None, missing=nan, monotone_constraints
=None,
             n_estimators=100, n_jobs=None, num_parallel_tree=None,
             predictor=None, random_state=None, ...)
```

In [75]:
```
y_pred_XGB = xgb_model.predict(X_test_pca)
```

In [76]:
```
XGB_Mse = mean_squared_error(y_test,y_pred_XGB)
XGB_Mse
```

Out[76]:
```
1.2461755934557024
```

In [77]:
```
XGB_MS = xgb_model.score(X_test_pca, y_test)
XGB_MS
```

Out[77]:
```
-0.13324490504994313
```

In [78]:
```
XGB_AR2 = adj_r_squared(y_test, y_pred_XGB, 2)
XGB_AR2
```

Out[78]:
```
-0.15197622579457049
```

In [79]:
```
#Decision Tree Regressor
from sklearn.tree import DecisionTreeRegressor
```

In [80]:
```
dt_model = DecisionTreeRegressor(max_depth=3)
```

In [81]:
```
dt_model.fit(X_train_pca, y_train)
```

Out[81]:
```
DecisionTreeRegressor(max_depth=3)
```

In [82]:
```
y_pred_dt = dt_model.predict(X_test_pca)
```

In [83]:
```
DT_Mse = mean_squared_error(y_test,y_pred_dt)
DT_Mse
```

Out[83]: 1.179678740136945

In [84]:
```
DT_MS = dt_model.score(X_test_pca, y_test)
DT_MS
```

Out[84]: -0.07277411696753022

In [85]:
```
DT_AR2 = adj_r_squared(y_test, y_pred_dt, 2)
DT_AR2
```

Out[85]: -0.09050592055377038

In [86]:
```
#Model Scores
print("Linear regression Model Score ", Lin_Reg_MS)
print("Ridge regression Model Score ", Ridge_MS)
print("Lasso regression Model Score ", Lasso_MS)
print("SVR regression Model Score ", SVR_MS)
print("XGB regression Model Score ", XGB_MS)
print("Decision Tree regression Model Score ", DT_MS)
```

```
Linear regression Model Score  -0.0673260467194885
Ridge regression Model Score  -0.06528916676786878
Lasso regression Model Score  -0.02213892633323189
SVR regression Model Score  -0.1106833195485768
XGB regression Model Score  -0.13324490504994313
Decision Tree regression Model Score  -0.07277411696753022
```

Code for UI :

Line wrap ☐

```
<!DOCTYPE html>
<html>
<head>
    <title>Calorie Prediction App</title>
    <style>
        body {
            background-color: orange;
        }

        h1 {
            text-align: center;
            color: white;
            font-size: 50px;
            margin-top: 50px;
        }

        form {
            margin-top: 50px;
            text-align: center;
        }

        label {
            font-size: 30px;
            color: white;
        }

        input[type="number"] {
            font-size: 25px;
            padding: 10px;
            border-radius: 5px;
            border: none;
            margin-bottom: 10px;
        }

        input[type="submit"] {
            font-size: 25px;
            padding: 10px;
            border-radius: 5px;
            border: none;
            background-color: white;
            color: black;
            cursor: pointer;
        }

        .result {
            text-align: center;
            font-size: 30px;
            color: white;
            margin-top: 50px;
        }
    </style>
</head>
<body>
    <h1>Calorie Prediction App</h1>

    <form id="calorie-form">
        <label for="age">Age (in years): </label>
        <input type="number" id="age" name="age" required>

        <br>

        <label for="height">Height (in cm): </label>
        <input type="number" id="height" name="height" required>

        <br>

        <label for="weight">Weight (in kg): </label>
        <input type="number" id="weight" name="weight" required>

        <br>

        <label for="duration">Duration (in minutes): </label>
        <input type="number" id="duration" name="duration" required>

        <br>

```

```
        <label for="heart_rate">Heart Rate (in bpm): </label>
        <input type="number" id="heart_rate" name="heart_rate" required>

        <br>

        <label for="body_temp">Body Temperature (in Celsius): </label>
        <input type="number" id="body_temp" name="body_temp" required>

        <br>

        <input type="submit" value="Predict Calories">
    </form>

    <div class="result" id="result"></div>

    <script>
        document.getElementById("calorie-form").addEventListener("submit", function(e) {
            e.preventDefault();

            var age = parseInt(document.getElementById("age").value);
            var height = parseInt(document.getElementById("height").value);
            var weight = parseInt(document.getElementById("weight").value);
            var duration = parseInt(document.getElementById("duration").value);
            var heart_rate = parseInt(document.getElementById("heart_rate").value);
            var body_temp = parseInt(document.getElementById("body_temp").value);

            var result = (10 * weight) + (6.25 * height) - (5 * age) + 5;
            result += (heart_rate * 0.5) - (duration * 0.7) + (body_temp * 0.1);

            document.getElementById("result").innerHTML = "Calories burned: " + result.toFixed(2);
        });
    </script>
</body>
</html>
```